

Iteris family 联合调度

合作者操作指南 (CLI + 目录约定)

Iteris 开发组 2026-06-20

目录

1	这份文档解决什么问题	2
2	前置条件	2
3	推荐目录结构	2
4	一次性初始化: <code>iteris family init</code>	3
4.1	方式 A: 显式声明 sibling (推荐, session 名可控)	3
4.2	方式 B: 自动收养子目录 / symlink	3
4.3	<code>--sibling</code> 字段说明	4
4.4	生成的 <code>FAMILY.json</code> (摘要)	4
5	日常命令	4
5.1	<code>iteris family status</code> — 看全族状态	4
5.2	<code>iteris family schedule</code> — 执行一次调度 tick	5
5.3	<code>iteris family start</code> — 手动启动	5
5.4	<code>iteris family run</code> — 后台 supervisor	5
5.5	<code>iteris family stop</code> — 停止 session	5
6	推荐工作流 (合作者 checklist)	6
7	当前能力边界 (2026-06-20)	6
8	常见问题	6
9	命令速查表	7

1 这份文档解决什么问题

当你需要并行推进一族相关的 **North-Star** 闭合题（例如 GECP Section 5 的 5.1/5.2/5.3，或 Problem 2.6/2.7/2.8）时，单独对每个 sibling 手工执行

```
iteris run ...
```

很快会失控：session 名不一致、不知道哪题在跑、忘了补开停掉的 loop。

iteris family 是 Iteris core 中的一组命令，用于：

- 在 **family 根目录** 维护一份 **.iteris/FAMILY.json** (sibling 列表、并行上限、family 总目标)；
- 用 **family status** 一眼看到各 sibling 的 **phase / tmux session / TASK_POOL frontier**；
- 用 **family schedule / family start** 按 **max_concurrent** 自动补开 sibling 的 /goal loop；
- 可选：**family run** 启动后台 supervisor，周期性执行 **schedule**。

[与 **iteris evolve** 的区别] **evolve** 管的是「一个 verified 结果 → 多个泛化方向 (direction pool)」。**family** 管的是「多个并行 sibling 各自闭合原题，必要时互相 cite verified 事实」。二者不要混用同一套 mental model。

2 前置条件

1. 已安装 Iteris CLI (**iteris --version** 可用)。代码更新后在本机执行：

```
cd /path/to/Iteris && bash scripts/deploy.sh
```

2. 已安装 **tmux** (family 通过 **tmux session** 管理 sibling run)。
3. 每个 **sibling** 必须是标准 Iteris 项目 (含 **iteris.toml**)，通常已有：
 - **.iteris/watchdog_goal.txt** (North-Star 约束文案)；
 - **tasks/TASK_POOL.json** (含 **task-align-*** 对齐任务)；
 - **docs/ANTI_DRIFT_ALIGNMENT.md** (gap 表与 freeze 规则，由 operator 维护)。
4. **Family 根目录** 本身不必是 Iteris 项目；它只是一个 wrapper 目录，下面放 sibling (子目录或 symlink)。

3 推荐目录结构

```
my-family/                                # family 根 (wrapper)
  .iteris/FAMILY.json                     # family 状态 (init 生成)
  .iteris/watchdog_goal.txt               # 可选: family 级总控 goal
  docs/FAMILY_OPERATOR.md                 # 人读: gap、round、session 约定
  docs/ANTI_DRIFT_ALIGNMENT.md
  gecp-5-1-quasi-optimality -> ../Iteris-GECP-evo-... # sibling (symlink 或子目录)
  gecp-5-2-windowed-matching -> ...
  gecp-5-3-fixed-sigma -> ...
```

每个 sibling 项目内会被写入 **.iteris/family.json** (指向 family 根与 sibling_id)，便于 monitor / 后续工具识别归属。

4 创建 family: iteris family new (推荐)

从零创建一族 workspace: 共享目录 + 每题一个完整 Iteris 项目 + 共享资源池路径。

4.1 方式 A: manifest 文件 (推荐)

模板见 `src/iteris/data/templates/family_manifest.example.json`。

```
iteris family new my-family --manifest family.json
```

manifest 中每个 sibling 需填写:

- sibling_id、path (子目录名)
- source (原题 markdown, 会复制进 sibling 的 sources/)
- north_star (写入 `.iteris/watchdog_goal.txt`)
- target_artifact、gaps、session、claim_prefix

4.2 方式 B: 命令行逐个 sibling

```
iteris family new my-family --goal "... " \
  --sibling "id=2.6,path=prob-26,source=sources/p26.md,\
north_star=Close problem 2.6...,target=results/p26/answer.md,gaps=FC-6"
```

4.3 共享资源池

```
# sibling 内 promote 后, 从 family 根 export:
iteris family export . --from prob-2-8-random-solve-codex \
  --fact-id fact:... --usable-by 2.6 --usable-by 2.7

iteris family pool          # 查看池
iteris tool memory search . --query "... " # 自动含 scope:family
```

每个 sibling 的 iteris run prompt 会自动注入 family closure 块 (可引用池内 verified 线索, 须本地 re-verify)。

在 family 根目录执行。

4.4 方式 A: 显式声明 sibling (推荐, session 名可控)

以 GECP Section 5 为例:

```
cd gecp-section5-family

iteris family init \
  --goal "North-Star full closure of GECP Section 5 (5.1/5.2/5.3)." \
  --sibling "id=5.1,path=gecp-5-1-quasi-optimality,\
session=iteris-gecp-s5-51,\
target=results/gecp-5-1-quasi-optimality/answer_northstar_verified.md,\
gaps=0C-5B" \
```

```
--sibling "id=5.2,path=gecp-5-2-windowed-matching,\
session=iteris-gecp-s5-52,\
target=results/gecp-5-2-windowed-matching/answer_northstar_verified.md,\
gaps=0C-5C" \
--sibling "id=5.3,path=gecp-5-3-fixed-sigma,\
session=iteris-gecp-s5-53,\
target=results/gecp-5-3-fixed-sigma/answer_northstar_verified.md,\
gaps=0C-5D" \
--max-concurrent 3
```

以 Problem 2.6–2.8 为例:

```
cd prob-2-6-7-8-family

iteris family init \
--goal "North-Star full closure of Problems 2.6, 2.7, 2.8." \
--sibling "id=2.6,path=prob-2-6-kappa-p-codex,session=iteris-prob-26,\
target=results/prob-2-6/answer_northstar_verified.md,gaps=FC-6" \
--sibling "id=2.7,path=prob-2-7-small-outliers-codex,session=iteris-prob-27,\
target=results/prob-2-7/answer_northstar_verified.md,gaps=FC-7" \
--sibling "id=2.8,path=prob-2-8-random-solve-codex,session=iteris-prob-28,\
target=results/prob-2-8/answer_northstar_verified.md,gaps=R2" \
--max-concurrent 3
```

4.5 方式 B: 自动收养子目录 / symlink

若 sibling 已是 family 根下的 Iteris 子目录或 symlink, 可省略 `--sibling`:

```
iteris family init --goal "..." --max-concurrent 2
```

`--adopt-symlinks` 为默认开启 (也会收养普通子目录中的 Iteris 项目)。

4.6 --sibling 字段说明

键	必填	含义
id	是	sibling 短名 (如 5.2、2.8)
path	是	相对 family 根的路径 (或 symlink 名)
session	否	tmux session 名; 省略则用 <code>iteris-<slug></code>
target	否	传给 <code>iteris run --target-artifact</code>
gaps	否	gap 标签, 用 分隔, 仅作 status 展示
priority	否	调度优先级 (整数, 越大越先补开)

4.7 生成的 FAMILY.json (摘要)

```
{
  "schema_version": "iteris.family_state.v0",
  "goal": "...",
```

```

"schedule": { "max_concurrent": 3, "stall_hours": 18.0 },
"policy": {
  "prefer_watchdog_goal": true,
  "allow_principled_stop": false,
  "claim_prefix": null
},
"siblings": [
  {
    "sibling_id": "5.2",
    "path": "gecp-5-2-windowed-matching",
    "session": "iteris-gecp-s5-52",
    "target_artifact": "results/.../answer_northstar_verified.md",
    "gaps": ["0C-5C"]
  }
]
}

```

5 日常命令

所有命令均在 **family 根目录** 执行（或显式传入路径）。加 `--json` 可得到机器可读输出，便于脚本集成。

5.1 iteris family status — 看全族状态

```

iteris family status
iteris family status --json

```

输出包含：

- **summary**: 几个 sibling closed、几个 running、还剩几个并发 slot；
- 每个 sibling 的 **phase**、**session_live**、**active_frontier**、ready 的 **task-align-*** 任务。

phase 含义（机械判定，供调度用）

phase	含义
open	尚无通过的 goal_success (或 policy 不允许的 terminal)
closed	verification ledger 中有通过的 goal_success
blocked_partial	有 principled_stop, 但 policy 默认不视为 family terminal
reduced	仅当 <code>--allow-principled-stop</code> 时, principled_stop 计为 terminal

closed 目前表示「ledger 里有过 goal_success」, 不一定等于 operator 定义的 Round 2 *full closure* (claim 前缀、gap 全关等)。详细数学进度仍以各 sibling 的 STATUS.md 与 answer_northstar_verified.md 为准。

5.2 iteris family schedule — 执行一次调度 tick

```
iteris family schedule --dry-run    # 只看会启动谁，不真开
iteris family schedule              # 对 open 且 session 未 live 的 sibling 启动 run
```

规则 (简化):

1. 统计 `phase=open` 且 `session` 已在跑的 sibling 数 R ;
2. 可用 `slot = max_concurrent - R;`
3. 在 `open` 且 `未 live` 的 sibling 中, 按 `priority / phase` 排序, 最多启动 `slot` 个;
4. 启动命令等价于在 sibling 目录执行 `iteris run`, goal 优先读 `.iteris/watchdog_goal.txt`。

5.3 iteris family start — 手动启动

```
iteris family start                # 同 schedule 逻辑
iteris family start --sibling 2.8  # 强制启动指定 sibling (忽略 max_concurrent)
iteris family start --dry-run --json
```

5.4 iteris family run — 后台 supervisor

```
iteris family run                  # detached tmux: iteris-family-<family-slug>
iteris family run --foreground --ticks 1 # 前台跑 1 个 tick (调试)
iteris family run --tick-seconds 900    # 默认每 15 分钟 schedule 一次
```

Supervisor 只负责补开 sibling session, 不修改 TASK_POOL, 也不注入 alignment 任务。

5.5 iteris family stop — 停止 session

```
iteris family stop --sibling 2.8  # 只停 2.8 的 tmux session
iteris family stop                # 停所有 sibling session (不含 supervisor)
iteris family stop --supervisor    # 停 family supervisor
```

6 推荐工作流 (合作者 checklist)

1. Pull 最新 Iteris 并 `deploy.sh` (若 CLI 行为与文档不符, 先查 `deploy skew: iteris run` 会提示)。
2. 进入 family 根; 若尚无 `FAMILY.json`, 按 §4 执行 `family init` (已有则跳过)。
3. `iteris family status`: 确认各 sibling `phase`、`session`、`ready align` 任务。
4. Operator 若做了 Round push (仍可能通过 `scripts/apply_alignment_*.py`), 先跑脚本再 `status`。
5. `iteris family schedule --dry-run` → 确认将要启动的 sibling → `family schedule` 或 `family start --sibling ...`。

6. 需要长期无人值守时: `iteris family run`; 结束用 `family stop --supervisor`。
7. attach 某个 sibling 的 agent loop:

```
iteris attach gecp-5-2-windowed-matching --session iteris-gecp-s5-52
```

8. 数学进度与 gap 闭合: 读 sibling 的 STATUS.md、docs/ANTI_DRIFT_ALIGNMENT.md; 不要只信 family 的 closed 计数。

7 当前能力边界 (2026-06-20)

- **已支持**: sibling 注册、status 聚合、watchdog goal 自动传入 `iteris run`、并发调度、supervisor loop。
- **尚未迁入 family CLI**: Round push 任务注入 / drift freeze(仍在 family 仓的 `apply_alignment_*.py`); 跨 sibling 依赖自动编排; claim 前缀硬门禁。
- **与 evolve 共存**: sibling 可以是 evolve 仓库; `iteris evolve stop` 与 `iteris family stop` 互不 cascade (与单题 `iteris stop` 语义一致)。

8 常见问题

Q: family status 显示 closed 但 watchdog 还要求 full closure? A: status 的 closed 是 ledger 机械判定。Round 1 partial 也可能产生 goal_success。以 artifact 标题前缀 (如 North-Star full closure of ...) 和 gap 表为准。

Q: schedule 说 nothing to start, 但某题明明没跑? A: 可能该 sibling 的 phase 已是 closed, 或 session 仍 live (含 closed 后未停的 loop)。用 `status --json` 看 `session_live`。

Q: init 报 cannot resolve sibling? A: 检查 path 是否指向含 `iteris.toml` 的目录; symlink 失效时可加 `evolve_dir` 到 `FAMILY.json` 手工编辑 (见 `family.py` 的 fallback)。

Q: 需要改 max_concurrent? A: 直接编辑 `.iteris/FAMILY.json` 的 `schedule.max_concurrent` (暂无单独 CLI 子命令)。

9 命令速查表

命令	作用
<code>iteris family new</code>	创建 family + N 个 sibling 项目 + 共享池目录
<code>iteris family export</code>	将 sibling verified fact 写入共享池
<code>iteris family pool</code>	列出共享池条目
<code>iteris family init</code>	注册已有 sibling layout (adopt symlink)

命令	作用
<code>iteris family status</code>	全族 phase / session / pool 概览
<code>iteris family schedule</code>	按并发上限补开 sibling run
<code>iteris family start</code>	手动启动 (可 <code>--sibling ID</code>)
<code>iteris family run</code>	后台 supervisor 周期 schedule
<code>iteris family stop</code>	停止 sibling 或 supervisor session

文档版本与 *Iteris* 源码同步维护于 *Iteris/docs/iteris-family-operator-guide.tex*。问题反馈请指向 *Iteris* 仓库 *issue* 或 *family operator*。